Business Analytics
Exam

NAMAN JAIN

ROLL NO - FT272022

SECTION 2

COURSE – Business Analytics

First, we read the csv file and the output shows the first 5 rows of the data

```
[8]: df = pd.read_csv('Sports Data_547601343_1788497867256.csv')
     df.head()
```

[8]:

	Game_number	Result	Avg_team_Age	Match_light_type	Match_format	Bowlers_in_team	Wicket_keeper_in_team	All_rounder_in_team	First_selection	Opponent	..
0	Game_1	1	18	Day	ODI	3	1	3	Bowling	Srilanka	.
1	Game_2	0	24	Day	T20	3	1	4	Batting	Zimbabwe	.
2	Game_3	1	24	Day and Night	T20	3	1	2	Bowling	Zimbabwe	.
3	Game_4	0	24	Day	ODI	2	1	2	Bowling	Kenya	.
4	Game_5	1	24	Night	ODI	1	1	3	Bowling	Srilanka	.

5 rows × 23 columns

Let us now check for the missing values.

Here, we have checked the null values and found that no null values are present

```
[9]: df.isnull().sum()
```

[9]:

```
Game_number              0
Result                   0
Avg_team_Age             0
Match_light_type         0
Match_format             0
Bowlers_in_team          0
Wicket_keeper_in_team    0
All_rounder_in_team      0
First_selection          0
Opponent                 0
Season                   0
Audience_number          0
Offshore                 0
Max_run_scored_1over     0
Max_wicket_taken_1over   0
Extra_bowls_bowled       0
Min_run_given_1over      0
Min_run_scored_1over     0
Max_run_given_1over      0
extra_bowls_opponent     0
player_highest_run       0
Players_scored_zero      0
player_highest_wicket    0
dtype: int64
```

There are no missing values in the dataset and we can go ahead with building the model.

There are no missing values in the dataset and we can go ahead with building the model.

Let us check the number of rows and the number of columns in the dataframe.

Checking for the number of rows and number of columns –

```
10]: print('The number of rows (observations) is:',df.shape[0],'\n''The number of columns(variables) is:',df.shape[1])

     The number of rows (observations) is: 2930
     The number of columns(variables) is: 23
```

The number of rows (observations) is: 2930

The number of columns(variables) is: 23

We have dropped 'Game_number', 'Wicket_keeper_in_team' variables and then again showed the first 5 rows

We will drop the ''Game_number', 'Wicket_keeper_in_team'' variable.

```
[12]: df=df.drop(['Game_number', 'Wicket_keeper_in_team'], axis=1)
      df.head()
```

[12]:

	Result	Avg_team_Age	Match_light_type	Match_format	Bowlers_in_team	All_rounder_in_team	First_selection	Opponent	Season	Audience_number	...	Max_run
0	1	18	Day	ODI	3	3	Bowling	Srilanka	Summer	9940	...	
1	0	24	Day	T20	3	4	Batting	Zimbabwe	Summer	8400	...	
2	1	24	Day and Night	T20	3	2	Bowling	Zimbabwe	Summer	13146	...	
3	0	24	Day	ODI	2	2	Bowling	Kenya	Summer	7357	...	
4	1	24	Night	ODI	1	3	Bowling	Srilanka	Summer	13328	...	

5 rows × 21 columns

Showing the data Types, Let us check the data types of each of the variables in the data.

```
[13]: df.dtypes
```

```
[13]: Result                  int64
      Avg_team_Age            int64
      Match_light_type          str
      Match_format              str
      Bowlers_in_team         int64
      All_rounder_in_team     int64
      First_selection           str
      Opponent                  str
      Season                    str
      Audience_number         int64
      Offshore                  str
      Max_run_scored_1over    int64
      Max_wicket_taken_1over  int64
      Extra_bowls_bowled      int64
      Min_run_given_1over     int64
      Min_run_scored_1over    int64
      Max_run_given_1over     int64
      extra_bowls_opponent    int64
      player_highest_run      int64
      Players_scored_zero     int64
      player_highest_wicket   int64
      dtype: object
```

This is to perform one hot encoding for Categorical Features -

```
[54]: # This line of code is to perform one hot encoding for Categorical Features
      X = pd.get_dummies( X, drop_first = False )
```

```
[55]: list(X.columns)
```

```
[55]: ['Avg_team_Age',
       'Bowlers_in_team',
       'All_rounder_in_team',
       'Audience_number',
       'Max_run_scored_1over',
       'Max_wicket_taken_1over',
       'Extra_bowls_bowled',
       'Min_run_given_1over',
       'Min_run_scored_1over',
       'Max_run_given_1over',
       'extra_bowls_opponent',
       'player_highest_run',
       'Players_scored_zero',
       'player_highest_wicket',
       'Match_light_type_Day',
       'Match_light_type_Day and Night',
       'Match_light_type_Night',
       'Match_format_20-20',
       'Match_format_ODI',
       'Match_format_T20',
       'Match_format_Test',
       'First_selection_Batting',
       'First_selection_Bowling',
       'Opponent_Australia',
       'Opponent_Bangladesh',
       'Opponent_England',
       'Opponent_Kenya',
       'Opponent_Pakistan',
       'Opponent_South Africa',
       'Opponent_Srilanka',
       'Opponent_West Indies',
       'Opponent_Zimbabwe',
       'Season_Rainy',
       'Season_Summer',
       'Season_Winter',
       'Offshore_No',
       'Offshore_Yes']
```

Now again printing the data types to ensure one-hot encoding ran perfectly –

```
[56]: X.dtypes
```

```
[56]: Avg_team_Age                    int64
      Bowlers_in_team                 int64
      All_rounder_in_team             int64
      Audience_number                 int64
      Max_run_scored_1over            int64
      Max_wicket_taken_1over          int64
      Extra_bowls_bowled              int64
      Min_run_given_1over             int64
      Min_run_scored_1over            int64
      Max_run_given_1over             int64
      extra_bowls_opponent            int64
      player_highest_run              int64
      Players_scored_zero             int64
      player_highest_wicket           int64
      Match_light_type_Day             bool
      Match_light_type_Day and Night   bool
      Match_light_type_Night           bool
      Match_format_20-20               bool
      Match_format_ODI                 bool
      Match_format_T20                 bool
      Match_format_Test                bool
      First_selection_Batting          bool
      First_selection_Bowling          bool
      Opponent_Australia               bool
      Opponent_Bangladesh              bool
      Opponent_England                 bool
      Opponent_Kenya                   bool
      Opponent_Pakistan                bool
      Opponent_South Africa            bool
      Opponent_Srilanka                bool
      Opponent_West Indies             bool
      Opponent_Zimbabwe                bool
      Season_Rainy                     bool
      Season_Summer                    bool
      Season_Winter                    bool
      Offshore_No                      bool
      Offshore_Yes                     bool
      dtype: object
```

We see that the data types of all the variables have been changed to either of ***int64*** or ***float64***.

We see that the data types of all the variables have been changed to either of int640 or float64.

Again using the head function to display first 5 rows –

```
[57]: #You shall notice One hot encoding converts catagorical variable (Levels) into seperate columns with binary values
      X.head()
```

[57]:

	Avg_team_Age	Bowlers_in_team	All_rounder_in_team	Audience_number	Max_run_scored_1over	Max_wicket_taken_1over	Extra_bowls_bowled	Min_run_given_1over
0	18	3	3	9940	13	3	0	2
1	24	3	4	8400	12	1	0	0
2	24	3	2	13146	14	4	0	0
3	24	2	2	7357	15	4	0	2
4	24	1	3	13328	12	4	0	0

5 rows × 37 columns

Before building the model we should split the data into Train and Test. We will thus build a model on the training data and use this model to predict on the test data.

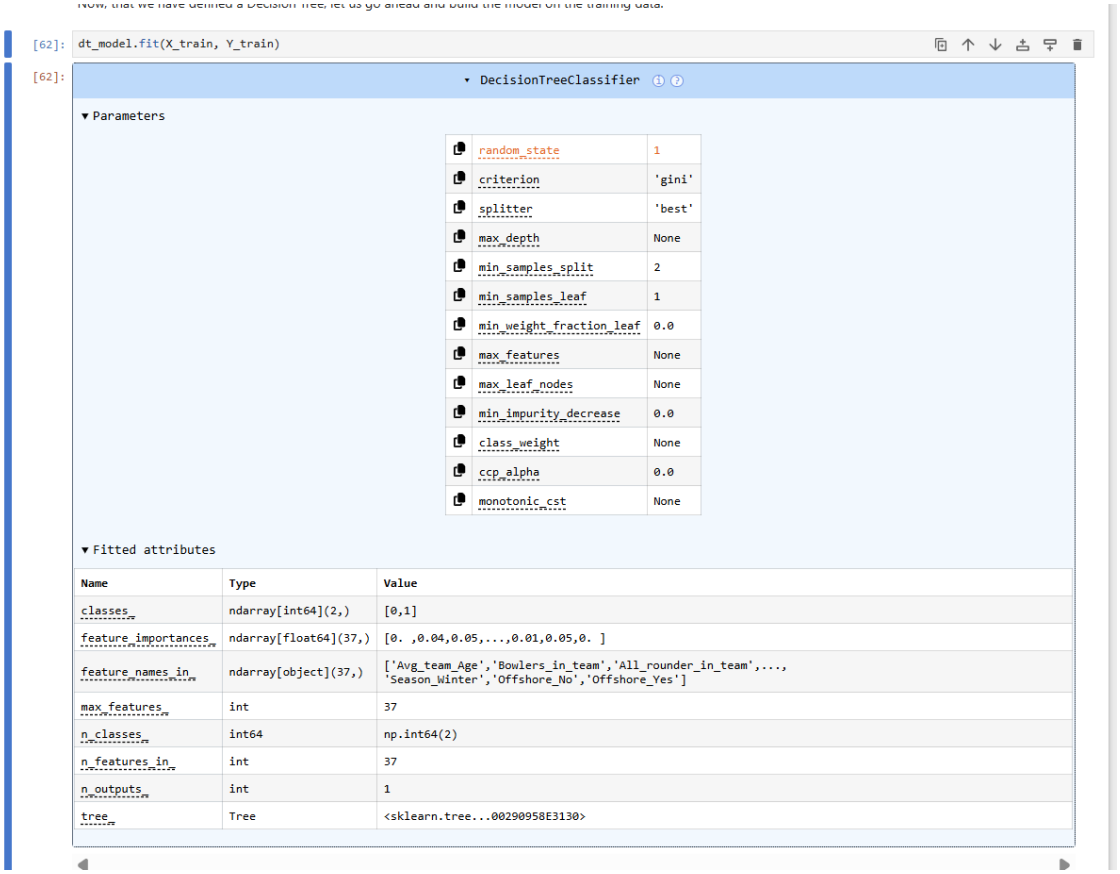
We will be doing a 70:30 split. 70% of the whole data will be used to train the data and then 30% of the data will be used for testing the model thus built.

In the code snippet we have defined a Decision Tree (which is to be used for classification problems) with the splitting criteria for each node as 'gini'. The 'random_state' parameter ensures that each time we run the code snippet the values remains the same.

In the code snippet which is uploaded, default values of 'min_samples_split' and 'min_samples_leaf' is taken as 2 and 1 respectively.

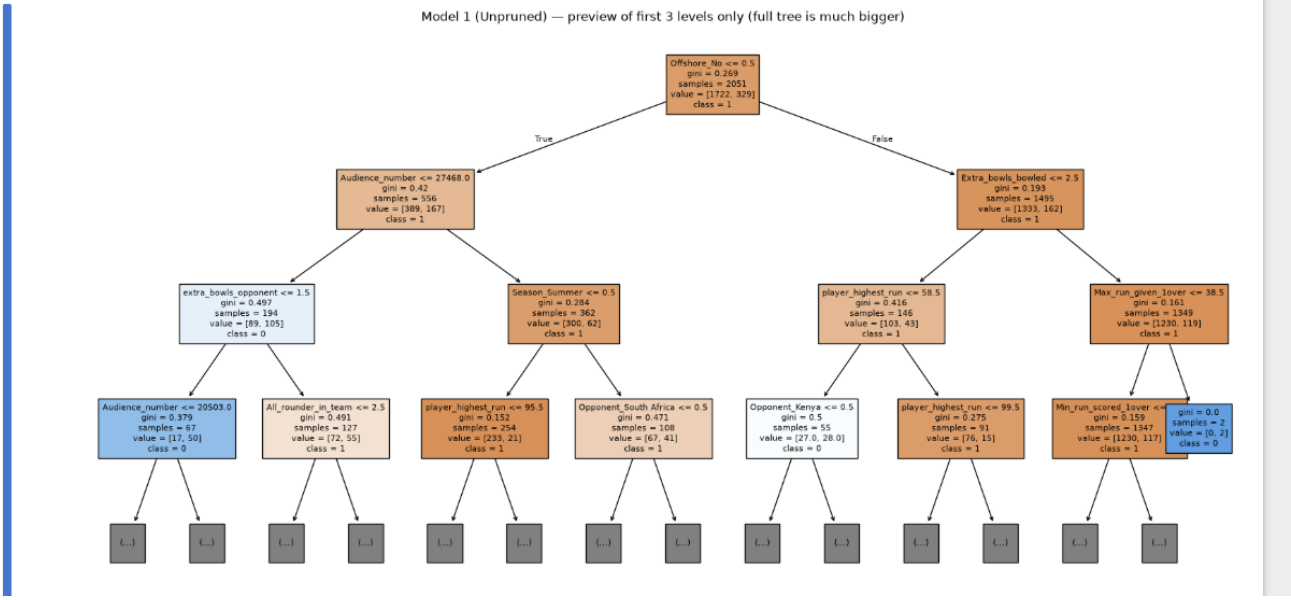
Now, that we have defined a Decision Tree, following is the Decision Tree Clasifier build the model on the training data.

This is the **initial** Decision Tree Classifier –



Visualising Unpruned Decision Tree –

```
[90]: plt.figure(figsize=(20,10))
      plot_tree(dt_model, feature_names=X_train.columns, class_names=[str(c) for c in Y.unique()],
                filled=True, max_depth=3, fontsize=8)
      plt.title("Model 1 (Unpruned) – preview of first 3 levels only (full tree is much bigger)")
      plt.show()
```



Now, that we have built the tree let us go ahead and visualize the tree to understand the various nuances of the Classification Tree that we just built.

Ranking the importance of the columns –

Now, that we have built the tree let us go ahead and visualize the tree to understand the various nuances of the Classification Tree that we just built.

```
[66]: pd.Series(dt_model.feature_importances_,index=X_train.columns).sort_values(ascending=False)
```

```
[66]: Audience_number                 0.191377
      player_highest_run              0.125105
      Extra_bowls_bowled              0.070732
      Max_run_scored_1over            0.055920
      Players_scored_zero             0.055458
      Offshore_No                     0.054086
      All_rounder_in_team             0.053254
      Season_Summer                   0.046588
      extra_bowls_opponent            0.044358
      Min_run_scored_1over            0.043347
      Bowlers_in_team                 0.041813
      Opponent_South Africa           0.037016
      Max_run_given_1over             0.026117
      Max_wicket_taken_1over          0.023845
      Opponent_Kenya                  0.019162
      player_highest_wicket           0.015306
      Min_run_given_1over             0.010920
      Match_light_type_Night          0.010672
      Match_light_type_Day and Night  0.009251
      Opponent_Srilanka               0.009232
      Opponent_Pakistan               0.008228
      Season_Winter                   0.008016
      Opponent_Australia              0.007570
      Match_format_T20                0.007477
      Match_format_ODI                0.006206
      Opponent_England                0.004638
      Season_Rainy                    0.003864
      Opponent_Zimbabwe               0.002662
      First_selection_Bowling         0.002413
      First_selection_Batting         0.002413
      Opponent_Bangladesh             0.001600
      Avg_team_Age                    0.001355
      Match_light_type_Day            0.000000
      Match_format_Test               0.000000
      Match_format_20-20              0.000000
      Opponent_West Indies            0.000000
      Offshore_Yes                    0.000000
      dtype: float64
```

From the above output, we can see that 'Audience_number' is the most important variable followed by 'player_highest_run' and so on.

From the above output, we can see that 'Audience_number' is the most important variable followed by 'player_highest_run' and so on.

Let us take a look at the overall accuracy of the train and test data using the model that we just built.

The decision for pruning was taken here -

Let us take a look at the overall accuracy of the train and test data using the model that we just built.

[67]:
```
#Train Data
dt_model.score(X_train,Y_train)
```

[67]:
```
1.0
```

[68]:
```
#Test Data
dt_model.score(X_test,Y_test)
```

[68]:
```
0.9328782707622298
```

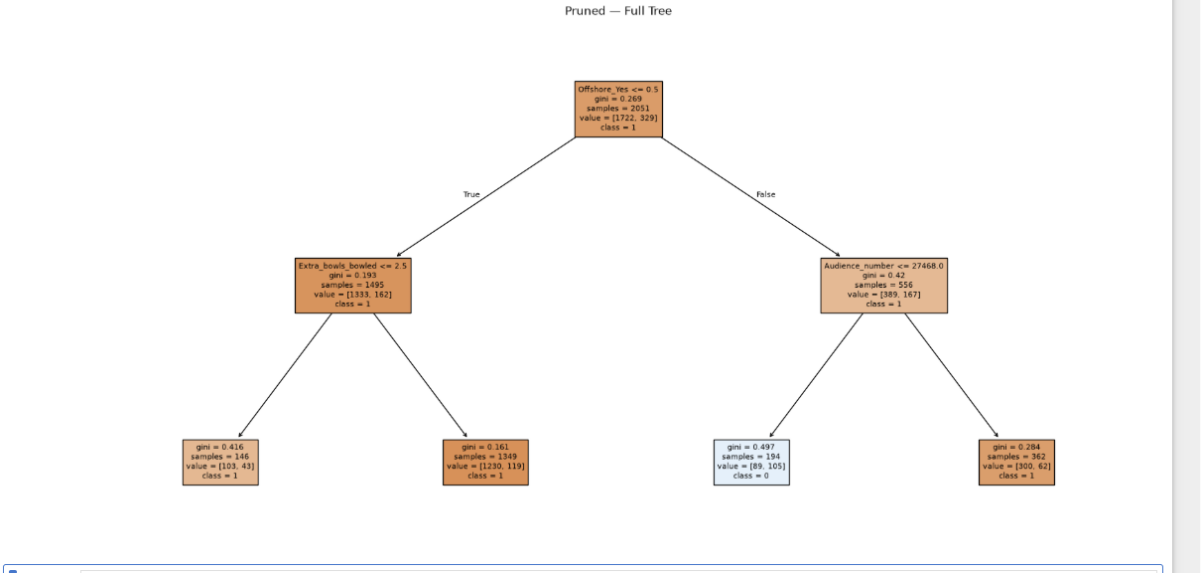
The accuracy on the Training Data is 100% and the accuracy on the Test Data is lesser substantially. The model has surely been overfitted. Thus, we need to prune or regularize the tree.

The accuracy on the Training Data is 100% and the accuracy on the Test Data is lesser substantially. The model has surely been overfitted. Thus, we need to prune or regularize the tree.

Pruned Decision Tree –

[71]:
```
'C:\\Users\\TeNeT'
```

[92]:
```
plt.figure(figsize=(20,10))
plot_tree(reg_dt_model, feature_names=X_train.columns, class_names=[str(c) for c in Y.unique()],
          filled=True, fontsize=8)
plt.title("Pruned – Full Tree")
plt.show()
```



Let us now go ahead and predict both the classes and the probability values on the test data using the Pruned/Regularized Decision Tree.

First we will evaluate the model on the **Training Data** -

True Negative: 1633

False Positives: 89

False Negatives: 224

True Positives: 105

[78]:
```
print(metrics.classification_report(Y_train,Y_train_predict_class))
```

```
              precision    recall  f1-score   support

           0       0.88      0.95      0.91      1722
           1       0.54      0.32      0.40       329

    accuracy                           0.85      2051
   macro avg       0.71      0.63      0.66      2051
weighted avg       0.83      0.85      0.83      2051
```

We have been able to predict 85% of the target variables correctly.

Let us check the confusion matrix for the **test data** –

True Negative: 696

False Positives: 39

False Negatives: 101

True Positives: 43

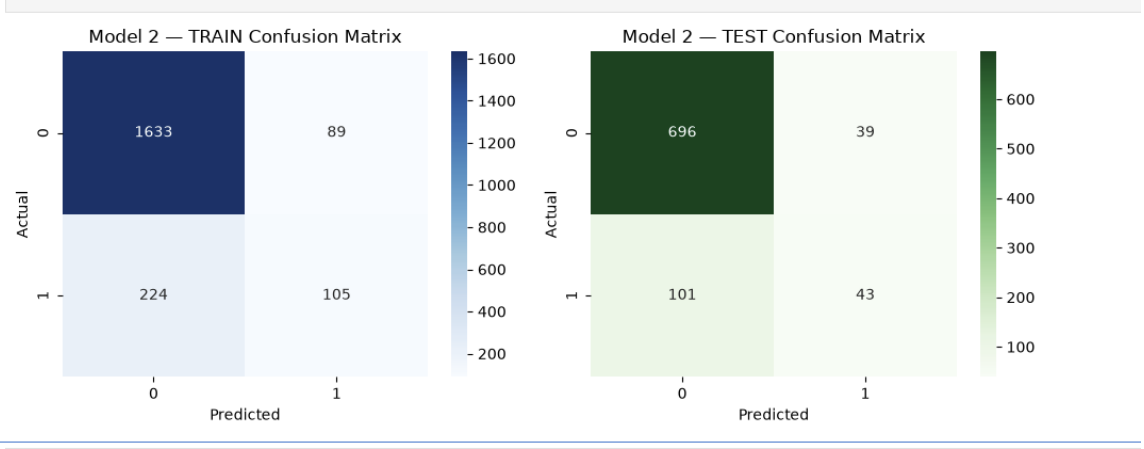
0]:
```
print(metrics.classification_report(Y_test,Y_test_predict_class))
```

```
              precision    recall  f1-score   support

           0       0.87      0.95      0.91       735
           1       0.52      0.30      0.38       144

    accuracy                           0.84       879
   macro avg       0.70      0.62      0.64       879
weighted avg       0.82      0.84      0.82       879
```

Heat Map for Visualisation –



FINAL INFERENCE -

Before building the model, the dataset was screened for variables that would add noise rather than genuine insight. This check removed 2 column(s) - Game_number, Wicket_keeper_in_team - because Game_number appeared to be a unique identifier for almost every record and therefore carried no repeatable pattern; Wicket_keeper_in_team held the same single value throughout the dataset and so contributed no information. Removing them ensures the model learns only from variables that carry real predictive information.

To arrive at a trustworthy Decision Tree, many different combinations of pruning settings were tested automatically, varying how deep the tree could grow and how many records were required before a node could split further. Each combination was scored on how well it balanced accuracy against overfitting, and the best-performing configuration required a maximum depth of 2 levels, at least 20 records before attempting a split, and at least 6 records in every resulting leaf.

The first, unrestricted tree (Model 1) reached a near-perfect 100.0% accuracy on the training data, but this proved to be misleading: on the test data, customers the model had never seen before, the accuracy fell to just 93.3%, a drop of 6.7% points. A fall of this size is a classic symptom of overfitting, meaning the tree had essentially memorised the training examples, including their noise and quirks, rather than learning a pattern that would hold up on new customers. After applying the pruning settings identified, the picture changed considerably. The pruned tree (Model 2) achieved 85%

accuracy on the training data and 84% on the test data; a gap of only 1 percentage points. This near-identical performance across both the training and test sets is strong evidence that the pruned model has learned a genuine, transferable relationship in the data rather than memorising individual records, making it the far more reliable model to trust with new customers going forward.

Beyond overall accuracy, it is worth examining the two different kinds of mistakes a classification model can make, since they carry very different costs in a real business setting. A Type 1 error (a False Positive) occurs when the model predicts the positive/'Yes' outcome for a case that would actually have turned out to be the negative/'No' outcome. For instance, in this context, this could mean turning away a genuine result of the match. A Type 2 error (a False Negative) is the opposite: the model predicts the negative/'No' outcome for a case that actually turns out to be the positive/'Yes' outcome. For instance, approving a loan for a customer who later defaults. On the test data, the unpruned model (Model 1) made 29 Type 1 errors (3.9% of all genuinely negative cases) and 30 Type 2 errors (20.8% of all genuinely positive cases). After pruning, Model 2 reduced these to 39 Type 1 errors (5.3%) and 101 Type 2 errors (70.1%). Neither error type improved meaningfully after pruning in this run, which suggests it may be worth revisiting the pruning parameters or reconsidering whether additional variables could help the model distinguish between the two classes more effectively.